



ORACLE

Requêtes multi-tables (jointures)

**Création et exploitation
de bases de données
420-315-AL**

Auteur :

Karim Mihoubi
Département d'informatique
Cégep André-Laurendeau
1111 Lapierre, Montréal (arr. LaSalle),
H8N 2J4
Tél: (514) 364-3320 [6162]
Karim.mihoubi@clairendeau.qc.ca



À voir...

ORACLE

- Algèbre relationnelle et jointures
 - Construire et exécuter une jointure naturelle
 - Construire et exécuter une jointure croisée
 - Construire et exécuter une jointure interne
 - Auto-jointures
 - Équijointure et inéquijointure
 - Construire et exécuter une jointure externes



Requêtes multi-tables

ORACLE

- Le *modèle relationnel* et la *normalisation des données* impliquent que les données sont réparties dans plusieurs tables
- Une *équijointure* utilise l'opérateur d'égalité dans la clause de jointure et compare généralement la *clé primaire* à la *clés étrangères*.
- Une *inéquijointure* fait intervenir tout type d'opérateur autre que l'égalité. La comparaison ne se fait généralement pas entre *clé primaire* et *clé étrangère*.
- **SQL** nous permet d'extraire des données à partir de plusieurs tables simultanément
- Ces requêtes sont liées (*jointes*) par les *données communes* entre deux tables



Jointure

ORACLE

- La « **Jointure** » est une opération qui consiste à effectuer un produit cartésien des enregistrements de deux tables pour lesquelles certaines valeurs correspondent. Le résultat de l'opération est une nouvelle table.
- Jointure dite SQL89[MAR 94]. Encore très utilisée, elle à l'avantage de laisser le soin au SGBD d'établir la meilleure stratégie d'accès pour optimiser les performances



Jointure

ORACLE

- La « **Jointure** » est une opération qui consiste à effectuer un produit cartésien des enregistrements de deux tables pour lesquelles certaines valeurs correspondent. Le résultat de l'opération est une nouvelle table.
- La table résultante est construite temporairement en fonction des prédicats spécifiés dans la requête.
- [Wikipédia](#)



Jointure croisée (*produit cartésien*)

ORACLE

- Le **produit cartésien** est l'opération fondamentale (cachée) sous-jacente à toutes les requêtes impliquant plus qu'une table
- Le **produit cartésien** de deux relations ***R1*** et ***R2*** est une relation ***R3*** dont les attributs est la ***concaténation des attributs*** des relations ***R1*** et ***R2*** et dont les uplets sont les ***concaténations de tous les uplets*** de ***R1*** et ***R2***



Jointure croisée (produit cartésien)

ORACLE

- La performance d'une requête, impliquant un **produit cartésien**, en terme de **temps de réponse** est **inversement proportionnelle** au **nombre de tables impliquées** dans la jointure



Jointure croisée (produit cartésien)

ORACLE

- La **jointure croisée** permet de réaliser un **produit cartésien**

Nous voulons voir toutes les combinaisons des vendeurs et des villes

```
SELECT VEN_NOM, BUR_VILLE
FROM EX1_VENDEUR_VEN
CROSS JOIN EX1_BUREAU_BUR;
```

Table EX1_VENDEUR_VEN → 10 uplets

Table EX1_BUREAU_BUR → 5 uplets

Combinaisons → $5 * 10 \rightarrow 50$ combinaisons possibles

Notez que le produit cartésien est une jointure INTERNE sans la clause ON...

VEN_NOM	BUR_VILLE
Samuel Cleroux	Vancouver
Robert Smith	Vancouver
Benoit Legault	Vancouver
Daniel Robert	Vancouver
Jacques Lacourse	Vancouver
Paul Guidon	Vancouver
Lionel Marchand	Vancouver
Suzanne Labonte	Vancouver
Nancy Fafard	Vancouver
Marie Juillet	Vancouver
Samuel Cleroux	Montreal
Robert Smith	Montreal
Benoit Legault	Montreal
Daniel Robert	Calgary
Jacques Lacourse	Calgary
Paul Guidon	Calgary
Lionel Marchand	Calgary
Suzanne Labonte	Calgary
Nancy Fafard	Calgary
Marie Juillet	Calgary

50 rows selected



Jointure interne

Équijointure

ORACLE

- Types de jointure qui permet de sélectionner des colonnes appartenant à **plus qu'une table** à partir d'une ou plusieurs conditions de jointures construites à l'aide **d'opérateur relationnelle**.
 - **Équijointure** si l'opérateur est =
 - **Inéquijointure** si autre opérateur

SELECT ...

FROM TABLE1 [INNER] JOIN TABLE2

ON TABLE1.COL1 = TABLE2.COL2

WHERE {autres conditions de recherche...}

Écriture Jointures SQL2 (SQL92)



Jointure interne

Équijointure

ORACLE

- Lorsque la requête questionne deux attributs de même nom appartenant à des tables différentes, il devient nécessaire de **préfixer** chaque nom d'attribut avec son nom de table
- En cas de conflit de nom, on peut aussi utiliser des **alias** pour donner un surnom à une table ou une colonne (**AS...**) → Le mot clé **AS** est optionnel...
- Il est alors plus facile d'écrire et de comprendre les requêtes de cette façon

SELECT ...

FROM TABLE1 AS T1 [INNER] JOIN TABLE2 AS T2
ON T1.COL1 = T2.COL2



Jointure relationnelle

Équijointure

ORACLE

- **ATTENTION À L'ANCIENNE ÉCRITURE!**

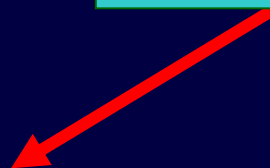
SELECT ...

FROM TABLE1, TABLE2

WHERE TABLE1.COL1 = TABLE2.COL2

AND {autres conditions de recherche...}

Condition de jointure



- Jointure dite SQL89[MAR 94]. Encore très utilisée, elle à l'avantage de laisser le soin au SGBD d'établir la meilleure stratégie d'accès pour optimiser les performances



Jointure interne

ORACLE

EX1_COMMANDE_COM

COM_NOCOMMANDE	COM_DATE_COMMANDE	COM_CLIENT	COM_REPRESENTANT	COM_USINE	COM_PRODUIT	COM_QUANTITE	COM_MONTANT
112961	05-12-17	2117	106 REI	2A44L		7	31500
113012	06-01-11	2111	105 ACI	41003		35	3745
112989	06-01-03	2101	106 FEA	114		6	1458
113051	06-02-10	2118	108 QSA	XK47		4	1420
112968	05-10-12	2102	101 ACI	41004		34	3978
113036	06-01-30	2107	110 ACI	4100Z		9	22500
113045	06-02-02	2112	108 REI	2A44R		10	45000
112963	05-12-17	2103	105 ACI	41004		28	3276
113013	06-01-14	2118	108 BIC	41003		1	652
113058	06-02-23	2108	109 FEA	112		10	1480

EX1_CLIENT_CLI

CLI_NOCLIENT	CLI_COMPAGNIE	CLI_VENDEUR	CLI_LIMITE CREDIT
2111	JCP Inc.	101	50000
2102	Premiere	101	65000
2103	Acme Mfg.	105	50000
2123	Cartier et Fils	102	40000
2107	Acura International	110	35000
2115	Smithson Corp.	101	20000
2101	Jones Mfg.	106	65000
2112	Zetacorp	108	50000
2121	QMA Assoc.	103	45000
2114	Orion Corp.	102	20000
2124	Pierre et Freres	107	40000
2108	Holmes et Poirot	109	55000
2117	J.P. Sinclair	106	35000

Relation clé primaire /
clé étrangère

- Pour chaque commande, affichez:
 - Le numéro de commande et son montant,
 - Le nom et la limite de crédit du client.



Jointure interne

ORACLE

```
SELECT COM_NOCOMMANDE, COM_MONTANT, CLI_COMPAGNIE, CLI_LIMITE_CREDIT
FROM EX1_COMMANDE_COM INNER JOIN EX1_CLIENT_CLI
ON COM_CLIENT = CLI_NOCLIENT
ORDER BY COM_NOCOMMANDE;
```

Les champs sélectionnés

Les tables

Le critère de jointure

COM_NOCOMMANDE	COM_MONTANT	CLI_COMPAGNIE	CLI_LIMITE_CREDIT
112961	31500	J.P. Sinclair	35000
112963	3276	Acme Mfg.	50000
112968	3978	Cie. Premiere	65000
112975	2100	JCP Inc.	50000
112979	15000	Orion Corp.	20000
112983	702	Acme Mfg.	50000
112987	27500	Acme Mfg.	50000
112989	1458	Jones Mfg.	65000



Jointure interne

ORACLE

Traitement manuel
de la requête

EX1_CLIENT_CLI

CLI_NOCLIENT	CLI_COMPAGNIE	CLI_VENDEUR	CLI_LIMITE_CREDIT
2111	JCP Inc.	101	50000
2102	Premiere	101	65000
2103	Acme Mfg.	105	50000
2114	Orion Corp.	102	20000
2124	Pierre et Freres	107	40000
2108	Holmes et Poirot	109	55000
2117	J.P. Sinclair	106	35000

EX1_COMMANDE_COM

COM_NOCOMMANDE	COM_DATE_COMMANDE	COM_CLIENT	COM_REPRESENTANT	COM_USINE	COM_PRODUIT	COM_QUANTITE	COM_MONTANT
112961	05-12-17	2117	106 REI	2A44L		7	31500
113012	06-01-11	2111	105 ACI	41003		35	3745
112969	06-01-03	2101	106 FEA	114		6	1458
11305	06-02-10	2118	108 QSA	XK47		4	1420

COM_NOCOMMANDE	COM_MONTANT	CLI_COMPAGNIE	CLI_LIMITE_CREDIT
112961	31500	J.P. Sinclair	35000
112963	3276	Acme Mfg.	50000
112968	3978	Cie. Premiere	65000



Jointure interne

ORACLE

```
SELECT cde.no_commande, cde.montant, clt.compagnie, clt.limite_credit  
FROM ex1_commande_com cde JOIN ex1_client_cli USING(code_client)  
ORDER BY cde.no_commande;
```

Possible uniquement si l'attribut
CODE_CLIENT existe dans les deux tables
sous ce nom

Les champs sélectionnés
Le critère de jointure
Les tables

COM_NOCOMMANDE	COM_MONTANT	CLI_COMPAGNIE	CLI_LIMITE_CREDIT
112961	31500	J.P. Sinclair	35000
112963	3276	Acme Mfg.	50000
112968	3978	Cie. Premiere	65000
112975	2100	JCP Inc.	50000
112979	15000	Orion Corp.	20000
112983	702	Acme Mfg.	50000
112987	27500	Acme Mfg.	50000
112989	1458	Jones Mfg.	65000



Jointure interne

ORACLE

```
SELECT cde.no_commande, cde.montant, clt.compagnie, clt.limite_credit  
FROM ex1_commande_com cde NATURAL JOIN ex1_client_cli  
ORDER BY cde.no_commande;
```

Possible uniquement si l'attribut
CODE_CLIENT existe dans les deux tables
sous ce nom

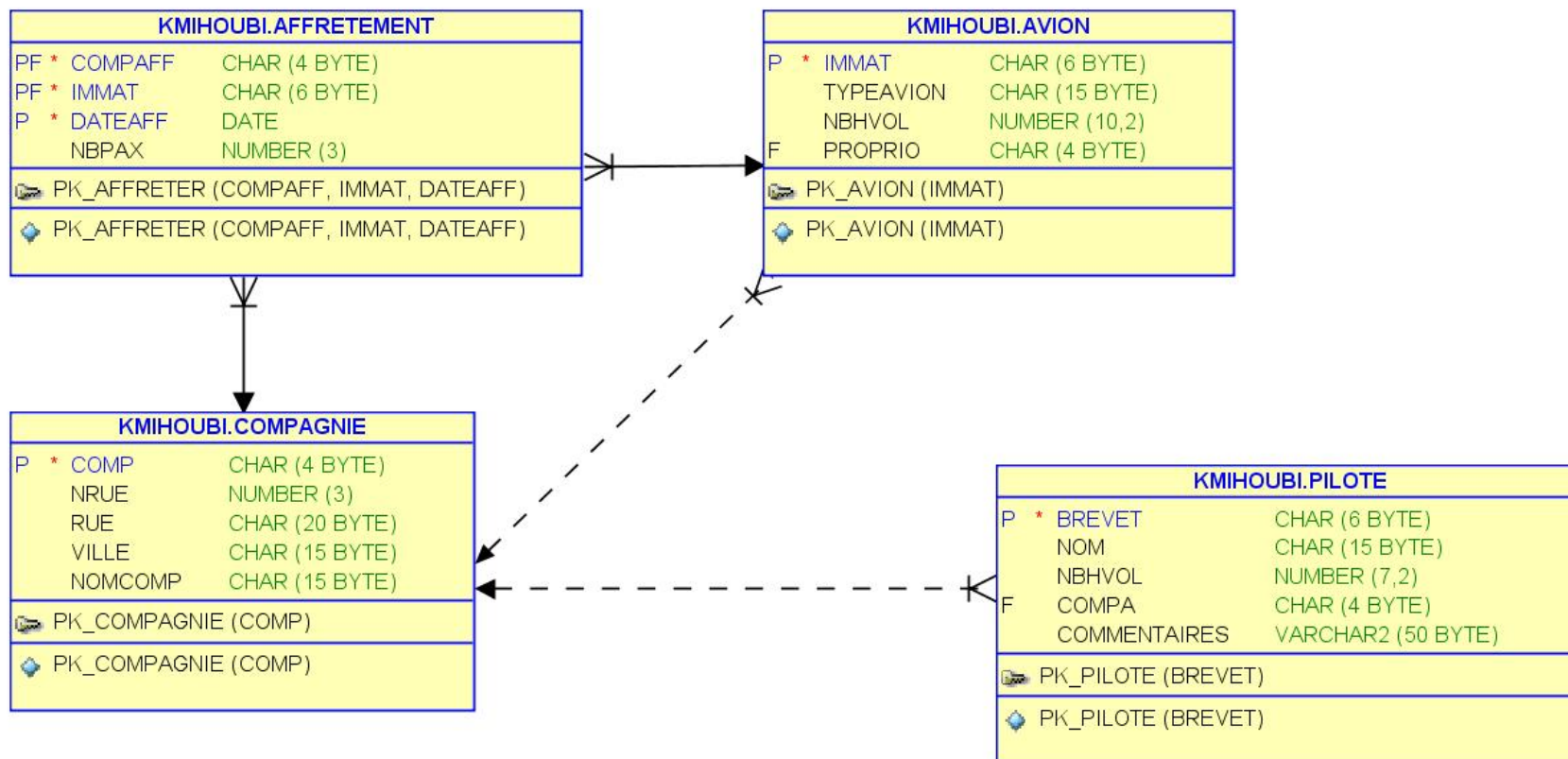
Les champs sélectionnés
Le critère de jointure
Les tables

COM_NOCOMMANDE	COM_MONTANT	CLI_COMPAGNIE	CLI_LIMITE_CREDIT
112961	31500	J.P. Sinclair	35000
112963	3276	Acme Mfg.	50000
112968	3978	Cie. Premiere	65000
112975	2100	JCP Inc.	50000
112979	15000	Orion Corp.	20000
112983	702	Acme Mfg.	50000
112987	27500	Acme Mfg.	50000
112989	1458	Jones Mfg.	65000



Jointure interne

ORACLE





Jointure interne

ORACLE

- **Deux styles d'écriture** : l'identité des pilotes de la compagnie 'AIR France' ayant plus de 500 heures de vols.

```
SELECT brevet, nom  
FROM Pilote, Compagnie  
WHERE comp = compa  
AND nomComp = 'Air France'  
AND nbhvol > 500;
```

```
SELECT brevet, nom  
FROM Compagnie  
JOIN Pilote ON comp = compa  
AND nomComp = 'Air France'  
AND nbhvol > 500;
```



Jointure interne

ORACLE

- **Deux styles d'écriture** : les coordonnées des compagnies qui embauchent des pilotes de plus de 950 heures de vol.

```
SELECT cpg.nomcomp, cpg.nrue, cpg.rue, cpg.ville  
FROM Pilote pil, Compagnie cpg  
WHERE cpg.comp = pil.compa  
AND pil.nbhvol > 950;
```

```
SELECT nomComp, nrue, ville  
FROM Compagnie  
JOIN Pilote ON comp = compa  
AND nbhvol > 950;
```



Relation parent – enfant

ORACLE

EX1_BUREAU_BUR

BUR_NOBUREAU	BUR_VILLE	BUR_REGION	BUR_GERANT	BUR_CIBLE	BUR_VENTE
22	Vancouver	Ouest	108	300000	186042
11	Montreal	Est	106	575000	692637
12	Toronto	Est	104	800000	735042
13	Ottawa	Est	105	350000	367911
21	Calgary	Ouest	108	725000	835915

EX1_BUREAU_BUR
(parent)



EX1_VENDEUR_VEN
(enfant)

EX1_VENDEUR_VEN

VEN_NOVENDEUR	VEN_NOM	VEN_AGE	VEN_BUREAU	VEN_TITRE	VEN_DA
101	Daniel Robert	45	12	Vendeur	95-10-20
102	Suzanne Labonte	48	21	Vendeur	05-12-10
103	Paul Guidon	29	12	Vendeur	06-03-01
104	Robert Smith	33	12	Gerant Ventes	01-05-19
105	Benoit Legault	37	13	Vendeur	03-02-12
106	Samuel Cleroux	52	11	Vice President	98-06-14
107	Nancy Fafard	49	22	Vendeur	98-11-14
108	Lionel Marchand	62	21	Gerant Ventes	99-10-12
109	Marie Juillet	31	11	Vendeur	01-10-12
110	Jacques Lacourse	41	(null)	Vendeur	02-01-13



Relation parent – enfant

ORACLE

```
-- AFFICHER LES NOMS DES VENDEURS AINSI QUE LEUR BUREAU ET LEUR REGION  
-- OU ILS TRAVAILLENT
```

```
SELECT VEN_NOM, BUR_VILLE, BUR_REGION  
FROM EX1_VENDEUR_VEN INNER JOIN EX1_BUREAU_BUR  
ON VEN_BUREAU = BUR_NOBUREAU  
ORDER BY BUR_REGION;
```

VEN_NOM	BUR_VILLE	BUR_REGION
-----	-----	-----
Samuel Cleroux	Montreal	Est
Robert Smith	Toronto	Est
Benoit Legault	Ottawa	Est
Daniel Robert	Toronto	Est
Paul Guidon	Toronto	Est
Marie Juillet	Montreal	Est
Lionel Marchand	Calgary	Ouest
Suzanne Labonte	Calgary	Ouest
Nancy Fafard	Vancouver	Ouest



Relation parent – enfant

ORACLE

EX1_VENDEUR_VEN

VEN_NOVENDEUR	VEN_NOM	VEN_AGE	VEN_BUREAU	VEN_TITRE	VEN_DA
101	Daniel Robert	45	12	Vendeur	95-10-20
102	Suzanne Labonte	48	21	Vendeur	05-12-10
103	Paul Guidon	29	12	Vendeur	06-03-01
104	Robert Smith	33	12	Gerant Ventes	01-05-19
105	Benoit Legault	37	13	Vendeur	03-02-12
106	Samuel Cleroux	52	11	Vice President	98-06-14
107	Nancy Fafard	49	22	Vendeur	98-11-14
108	Lionel Marchand	62	21	Gerant Ventes	99-10-12
109	Marie Jurek	31	11	Vendeur	01-10-12
110	Jacques Lacours	41	(null)	Vendeur	02-01-13

EX1_VENDEUR_VEN
(parent)



EX1_BUREAU_BUR
(enfant)

EX1_BUREAU_BUR

BUR_NOBUREAU	BUR_VILLE	BUR_REGION	BUR_GERANT	BUR_CIBLE	BUR_VENTE
22	Vancouver	Ouest	108	300000	186042
11	Montreal	Est	106	575000	692637
12	Toronto	Est	104	800000	735042
13	Ottawa	Est	105	350000	367911
21	Calgary	Ouest	108	725000	835915



Relation parent – enfant

ORACLE

```
-- AFFICHER LES BUREAUX AINSI QUE LE NOM ET LE TITRE  
-- DE LEUR GERANT
```

```
SELECT BUR_VILLE, VEN_NOM, VEN_TITRE  
FROM EX1_BUREAU_BUR INNER JOIN EX1_VENDEUR_VEN  
ON BUR_GERANT = VEN_NOVENDEUR  
ORDER BY BUR_VILLE;
```

BUR_VILLE	VEN_NOM	VEN_TITRE
-----	-----	-----
Calgary	Lionel Marchand	Gerant Ventes
Montreal	Samuel Cleroux	Vice President
Ottawa	Benoit Legault	Vendeur
Toronto	Robert Smith	Gerant Ventes
Vancouver	Lionel Marchand	Gerant Ventes



Jointure interne avec critère

ORACLE

```
-- AFFICHER LES BUREAUX, LE NOM DE LEUR GERANT ET LEUR TITRE  
-- POUR LES BUREAUX DONT LA CIBLE EST PLUS DE $600 000
```

```
SELECT BUR_VILLE, VEN_NOM, VEN_TITRE  
FROM EX1_BUREAU_BUR INNER JOIN EX1_VENDEUR_VEN  
ON BUR_GERANT = VEN_NOVENDEUR  
WHERE BUR_CIBLE > 600000  
ORDER BY BUR_VILLE;
```

Critère de sélection

BUR_VILLE	VEN_NOM	VEN_TITRE
Calgary	Lionel Marchand	Gerant Ventes
Toronto	Robert Smith	Gerant Ventes



Jointure interne sur plusieurs attributs

ORACLE

```
-- AFFICHER LES COMMANDES, LEUR MONTANT ET LA  
-- DESCRIPTION DES PRODUITS  
  
SELECT COM_NOCOMMANDE, COM_MONTANT, PRO_DESCRIPTION  
FROM EX1_COMMANDE_COM INNER JOIN EX1_PRODUIT_PRO  
ON COM_USINE = PRO_NOUSINE  
AND COM_PRODUIT = PRO_NOPRODUIT  
ORDER BY COM_NOCOMMANDE;
```

COM_NOCOMMANDE	COM_MONTANT	PRO_DESCRIPTION
112961	31500	Charniere Gauche
112963	3276	Support Moteur no.4
112968	3978	Support Moteur no.4
112975	2100	Cheville Charniere
112979	15000	Installateur Moteur
112983	702	Support Moteur no.4



Jointure interne sur plusieurs tables

ORACLE

- On peut aussi réaliser des *jointures internes* sur plusieurs tables distinctes

SELECT ...

FROM TABLE1

[INNER] JOIN TABLE2 ON { condition jointure #1 }

[INNER] JOIN TABLE3 ON { condition jointure #2 }

[INNER] JOIN TABLE4 ON { condition jointure #3 }

...

WHERE {autres conditions de recherche...}



Jointure interne sur plusieurs tables

ORACLE

EX1_VENDEUR_VEN

VEN_NOVENDEUR	VEN_NOM	VEN_AGE	VEN_BUREAU	VEN_TITRE	VEN_DA
101	Daniel Robert	45	12	Vendeur	95-10-20
102	Stéphanne Labonte	48	21	Vendeur	05-12-10
103	Paul Gidon	29	12	Vendeur	06-03-01
104	Robert Smith	33	12	Gerant Vent	01-05-19

EX1_CLIENT_CLI

CLI_NOCLIENT	CLI_COMPAGNIE	CLI_VENDEUR	CLI_LIMITE_CREDIT
2111	JCP Inc.	101	50000
2102	Premiere	101	65000
2103	Acme Mfg	105	50000

EX1_COMMANDE_COM

COM_NOCOMMANDE	COM_DATE_COMMANDE	COM_CLIENT	COM_REPRESENTANT	COM_USINE	COM_PRODUIT	COM_QUANTITE	COM_MONTANT
112961	05-12-17	2117	106 REI	2A44L		7	31500
113012	06-01-11	2111	105 ACI	41003		35	3745
112989	06-01-03	2101	106 FEA	114		6	1458
113051	06-02-10	2118	108 QSA	XK47		4	1420



Jointure interne avec plusieurs tables

ORACLE

```
-- POUR LES COMMANDES DE PLUS DE $25 000, AFFICHER LE NUMÉRO DE COMMANDE  
-- LE NOM DU CLIENT, LE MONTANT DE LA COMMANDE ET LE NOM DE SON VENDEUR  
  
SELECT COM_NOCOMMANDE, COM_MONTANT, CLI_COMPAGNIE, VEN_NOM  
FROM EX1_COMMANDE_COM  
      INNER JOIN EX1_CLIENT_CLI ON COM_CLIENT = CLI_NOCLIENT  
      INNER JOIN EX1_VENDEUR_VEN ON CLI_VENDEUR = VEN_NOVENDEUR  
WHERE COM_MONTANT > 25000  
ORDER BY COM_NOCOMMANDE;
```

COM_NOCOMMANDE	COM_MONTANT	CLI_COMPAGNIE	VEN_NOM
112961	31500	J.P. Sinclair	Samuel Cleroux
112987	27500	Acme Mfg.	Benoit Legault
113045	45000	Zetacorp	Lionel Marchand
113069	31350	Chen et Associates	Paul Guidon



Jointure interne avec plusieurs tables

ORACLE

```
-- POUR LES COMMANDES DE PLUS DE $25 000, AFFICHER LE NUMÉRO DE COMMANDE  
-- LE NOM DU CLIENT, LE MONTANT DE LA COMMANDE, LE NOM DE SON VENDEUR  
-- ET LA VILLE OU TRAVAILLE CE VENDEUR
```

```
SELECT COM_NOCOMMANDE, COM_MONTANT, CLI_COMPAGNIE, VEN_NOM, BUR_VILLE  
FROM EX1_COMMANDE_COM  
    INNER JOIN EX1_CLIENT_CLI ON COM_CLIENT = CLI_NOCLIENT  
    INNER JOIN EX1_VENDEUR_VEN ON CLI_VENDEUR = VEN_NOVENDEUR  
    INNER JOIN EX1_BUREAU_BUR ON VEN_BUREAU = BUR_NOBUREAU  
WHERE COM_MONTANT > 25000  
ORDER BY COM_NOCOMMANDE;
```

COM_NOCOMMANDE	COM_MONTANT	CLI_COMPAGNIE	VEN_NOM	BUR_VILLE
112961	31500	J.P. Sinclair	Samuel Cleroux	Montreal
112987	27500	Acme Mfg.	Benoit Legault	Ottawa
113045	45000	Zetacorp	Lionel Marchand	Calgary
113069	31350	Chen et Associes	Paul Guidon	Toronto



Auto-jointures (relation unaire)

ORACLE

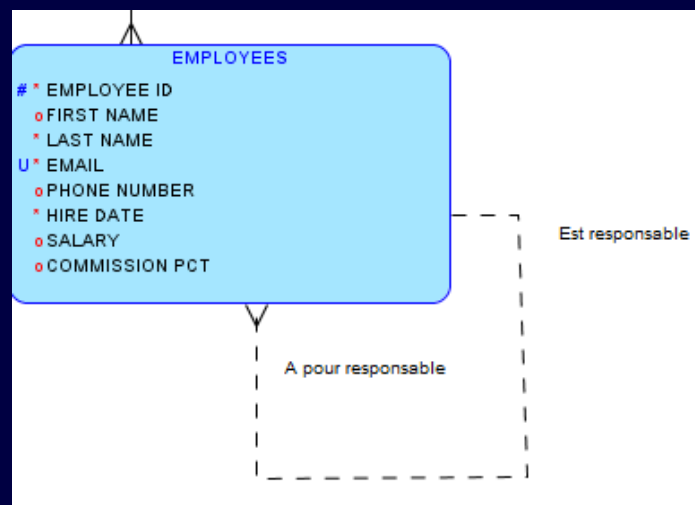
- En modélisant les données, il est parfois nécessaire de montrer une entité en relation avec elle-même.
- Exemple, un employé peut également être un responsable.
- Nous avons montré cela en utilisant la relation récursive ou "oreille de cochon"
- Il y a **auto-jointure** lorsqu'on doit comparer :
 - Le contenu d'un champ d'une **table** avec celui d'un autre champ de la **même table**



Auto-jointures (relation unaire)

ORACLE

- La norme **SQL** ne permet pas de citer une même table deux fois dans un **SELECT**
- La solution réside dans l'utilisation d'un alias de table, ce qui permet d'utiliser 2 copies de la même table





Auto-jointures (relation unaire)

ORACLE

On veut trouver les noms des gérants de tous les employés
(table **EX1_VENDEUR_VEN**)

EX1_VENDEUR_VEN

VEN_NOVENDEUR	VEN_NOM	VEN_AGE
101	Daniel Robert	45
102	Suzanne Labonte	48
103	Paul Guidon	29
104	Robert Smith	33
105	Benoit Legault	37
106	Samuel Cleroux	52
107	Nancy Fafard	49

EX1_VENDEUR_VEN

VEN_NOVENDEUR	VEN_NOM	VEN_AGE	VEN_BOVENDEUR	VEN_TITRE	VEN_DATE_EMBARCQUE	VEN_GERANT	VEN_QUOTA
108	101 Daniel Robert	45	12 Vendeur	95-10-20		104	300000
109	102 Suzanne Labonte	48	21 Vendeur	05-12-10		108	350000
110	103 Paul Guidon	29	12 Vendeur	06-03-07		104	275000
	104 Robert Smith	33	12 Gerant Ventres	01-05-19		106	200000
	105 Benoit Legault	37	13 Vendeur	03-02-12		104	350000
	106 Samuel Cleroux	52	11 Vice President	98-06-14		(null)	275000
	107 Nancy Fafard	49	22 Vendeur	98-11-14		108	300000
	108 Lionel Marchand	62	21 Gerant Ventres	99-10-12		106	350000
	109 Marie Juillet	31	11 Vendeur	01-10-12		106	300000
	110 Jacques Lacourse	41	(null) Vendeur	02-01-13		101	(null)



Auto-jointures (relation unaire)

ORACLE

Alias de colonne

```
-- AFFICHER LES NOMS DES GERANTS DE TOUS LES EMPLOYES  
SELECT V1.VEN_NOM AS VENDEUR, V2.VEN_NOM AS GERANT  
FROM EX1_VENDEUR_VEN V1 INNER JOIN EX1_VENDEUR_VEN V2  
ON V1.VEN_GERANT = V2.VEN_NOMVENDEUR;
```

VENDEUR	GERANT
Robert Smith	Samuel Cleroux
Benoit Legault	Robert Smith
Daniel Robert	Robert Smith
Jacques Lacourse	Daniel Robert
Paul Guidon	Robert Smith
Lionel Marchand	Samuel Cleroux
Suzanne Labonte	Lionel Marchand
Nancy Fafard	Lionel Marchand
Marie Juillet	Samuel Cleroux

Alias de table



Auto-jointures (relation unaire)

ORACLE

Autre
exemple

```
-- AFFICHER LES NOMS DES VENDEURS QUI ONT UN QUOTA  
-- PLUS ELEVE QUE CELUI DE LEUR GERANT  
  
SELECT V1.VEN_NOM AS VENDEUR, V1.VEN_QUOTA AS QUOTA_VENDEUR,  
       V2.VEN_NOM AS GERANT, V2.VEN_QUOTA AS QUOTA_GERANT  
FROM EX1_VENDEUR_VEN V1 INNER JOIN EX1_VENDEUR_VEN V2  
ON V1.VEN_GERANT = V2.VEN_NOVENDEUR  
AND V1.VEN_QUOTA > V2.VEN_QUOTA;
```

VENDEUR	QUOTA_VENDEUR	GERANT	QUOTA_GERANT
Benoit Legault	350000	Robert Smith	200000
Daniel Robert	300000	Robert Smith	200000
Paul Guidon	275000	Robert Smith	200000
Lionel Marchand	350000	Samuel Cleroux	275000
Marie Juillet	300000	Samuel Cleroux	275000



Règles de traitement des requêtes multi-tables

ORACLE

1. Si la requête contient une **UNION**, appliquer les étapes 2 à 5 à chaque partie de la requête pour générer les résultats respectifs
2. Former le **produit cartésien** des tables nommées dans la clause **FROM**. **Appliquer la jointure interne en utilisant le prédicat ON**
3. S'il existe une clause **WHERE**, appliquer la condition de recherche sur chaque uplet de la table générée **en ne retenant que les uplets pour lesquels** la condition est **VRAIE**



Règles de traitement des requêtes multi-tables

ORACLE

4. Pour **chaque uplet restant**, calculer la valeur de chaque attribut mentionné dans la clause **SELECT** pour générer chaque ligne de résultats
 - Pour une référence simple, utiliser la valeur de l'attribut de l'uplet courant
5. Si la clause **DISTINCT** est spécifiée, **éliminer tout uplet dupliqué**
6. Si l'énoncé contient une **UNION**, fusionner les résultats obtenus pour chaque partie de la requête, en **éliminant les uplets dupliqués** à moins que la clause **UNION ALL** ait été spécifiée
7. Si la clause **ORDER BY** est présente, **trier les uplets selon les critères spécifiés**



Règles de traitement des requêtes multi-tables

ORACLE

```
-- AFFICHER LE NOM DE LA COMPAGNIE ET TOUTES LES COMMANDES  
-- (#COMMANDE, MONTANT) DU CLIENT 2103
```

4 **SELECT** CLI_COMPAGNIE, COM_NOCOMMANDE, COM_MONTANT
2 **FROM** EX1_CLIENT_CLI **INNER JOIN** EX1_COMMANDE_COM
ON CLI_NOCLIENT = COM_CLIENT
3 **WHERE** CLI_NOCLIENT = 2103
7 **ORDER BY** COM_NOCOMMANDE;

CLI_COMPAGNIE	COM_NOCOMMANDE	COM_MONTANT
-----	-----	-----
Acme Mfg.	112963	3276
Acme Mfg.	112983	702
Acme Mfg.	112987	27500
Acme Mfg.	113027	4104



Règles de traitement des requêtes multi-tables

ORACLE

1. La clause **FROM** génère toutes les combinaisons possibles (produit cartésien) entre les uplets des deux tables
 - **Table EX1_CLIENT_CLI** → 21 uplets de 4 attributs
 - **Table EX1_COMMANDE_COM** → 30 uplets de 8 attributs
 - **Table résultante** → 630 uplets de 12 attributs (**cross join**)
 - L'application du **prédicat ON... de la jointure** ramène à 30 le nombre d'uplets de 12 attributs (**inner join**)
2. La clause **WHERE** extrait seulement les uplets (provenant de la **table résultante**) où la spécification (**CLI_NOCLIENT = 2103**) est exacte
 - 4 uplets de 12 attributs
3. La clause **SELECT** choisit les attributs spécifiés (**CLI_COMPAGNIE, COM_NOCOMMANDE, COM_MONTANT**) de chaque uplet restant
4. La clause **ORDER BY** trie les 4 uplets obtenus sur l'attribut **COM_NOCOMMANDE** pour générer le résultat final



Règles de traitement des requêtes multi-tables

ORACLE

Attention!

- Aucun **SGBD** ne traite les requêtes **SQL** de cette façon systématiquement!
- Cette façon de faire est théorique et permet de comprendre la démarche d'analyse d'une requête multi-tables
- Le **SGBD** utilise des méthodes d'optimisation pour trouver la méthode la plus efficace pour traiter la requête à l'aide de ...
 - Création d'un plan d'exécution
 - Méthodes d'optimisation
- Références pour **Oracle** →
 - **Oracle Concepts** → Chapitre 18



Jointure externe

ORACLE

- La **jointure externe** permet de sélectionner tous les uplets de deux tables pour lesquels la condition de jointure est vraie, **ET EN PLUS** tous les uplets de l'une ou de l'autre de ces tables **qui n'ont pas de lien** avec l'autre table
- Lorsque deux tables sont en jointure externe, une table est « **dominante** » par rapport à l'autre (**subordonnée**).
- Ce type de **jointure** est nécessaire lorsque:
 - La correspondance des uplets ne peut être établie entre les deux tables car l'information n'existe pas (valeur 'NULL' ou absence de valeur)
 - La correspondance des uplets ne satisfait pas au prédicat de jointure



Jointure externe

ORACLE

- *Hypothèse du monde clos*
 - *Cette hypothèse considère que tout ce qui n'est pas connu est réputé FAUX*
 - Si nous demandons à un vendeur le nom des clients qui habitent à Montréal, il se peut que certains clients de Montréal n'apparaissent pas dans la liste car leur adresse est inconnue
→ *Perte potentielle d'information*
 - On peut également supposer que parmi les clients qui n'ont pas fournis d'adresse existent des clients de Montréal.
 - La *jointure externe* nous donne la possibilité d'inclure dans la liste, les cas non connus...



Règles de traitement d'une jointure externe complète

ORACLE

1. Réaliser la *jointure interne* entre les deux tables
2. Pour **chaque uplet** de la **1^{ère} table** qui ne correspond pas à **aucun uplet** de la **2^e table**, ajouter-le aux résultats en attribuant des **NULLS** aux données manquantes de la **2^e table**
3. Pour **chaque uplet** de la **2^e table** qui ne correspond pas à **aucun uplet** de la **1^{ère} table**, ajouter-le aux résultats en attribuant des **NULLS** aux **données manquantes de la 1^{ère} table**



Jointure externe

ORACLE

- La **jointure externe « gauche »** permet de conserver tous les uplets de la table de gauche (**table dominante**) qui n'ont aucune correspondance avec ceux de la table de droite (**table subordonnée**) (*Règles 1 et 2*)
- La **jointure externe « droite »** permet de conserver tous les uplets de la table de droite (**table principale**) qui n'ont aucune correspondance avec ceux de la table de gauche (**table secondaire**) (*Règles 1 et 3*)
- La **jointure externe « complète »** ou **« bilatérale »** permet de conserver tous les uplets de la table gauche qui n'ont aucune correspondance avec ceux de la table de droite et tous les uplets de la table de droite qui n'ont aucune correspondance avec ceux de la table de gauche (*Règles 1, 2 et 3*)



Jointure externe (gauche)

ORACLE

- La liste des compagnies et leurs pilotes, même les compagnies n'ayant pas de pilote.

```
SELECT  cpg.nomComp, pil.brevet, pil.nom
FROM    Pilote pil, Compagnie cpg
WHERE   pil.compa(+)    = cpg.comp;
```

```
SELECT  nomComp, brevet, nom
FROM    Compagnie LEFT OUTER JOIN Pilote
        ON comp = compa;
```

	NOMCOMP	BREVET	NOM
1	Air France	PL-1	Amélie Sulpice
2	Air France	PL-2	Thomas Sulpice
3	Singapore AL	PL-3	Paul Soutou
4	Air Nul2	(null)	(null)
5	Air Null	(null)	(null)
6	Castanet Air	(null)	(null)



Jointure externe (droite)

ORACLE

- La liste des compagnies et leurs pilotes, même les compagnies n'ayant pas de pilote.

```
SELECT  cpg.nomComp, pil.brevet, pil.nom  
FROM    Pilote pil, Compagnie cpg  
WHERE   cpg.comp      = pil.compa(+);
```

```
SELECT  nomComp, brevet, nom  
FROM    Pilote RIGHT OUTER JOIN Compagnie  
        ON comp = compa;
```



Jointure externe (bilatérale)

ORACLE

- La jointure externe bilatérale se programme en faisant l'union de deux jointures externes unilatérales, en plaçant alternativement le symbole « (+) ».

```
SELECT  cpg.nomComp, pil.brevet, pil.nom
FROM    Pilote pil, Compagnie cpg
WHERE   cpg.comp(+)      = pil.compa
UNION
SELECT  cpg.nomComp, pil.brevet, pil.nom
FROM    Pilote pil, Compagnie cpg
WHERE   cpg.comp      = pil.compa(+)
ORDER BY 1
```

```
SELECT  nomComp, brevet, nom
FROM    Compagnie FULL OUTER JOIN Pilote
ON      comp = compa;
```

1

	NOMCOMP	BREVET	NOM
1	Singapore AL	PL-3	Paul Soutou
2	Air France	PL-2	Thomas Sulpice
3	Air France	PL-1	Amélie Sulpice
4	Castanet Air	(null)	(null)
5	(null)	PL-5	Michel Castain



Jointure externe

ORACLE

- *Cherchons les villes où se retrouvent les vendeurs...*

On débute en regardant le contenu de la table **EX1_VENDEUR_VEN**; quels sont les numéros de bureau de chaque représentant?

N.B. Jacques Lacourse n'a pas de bureau

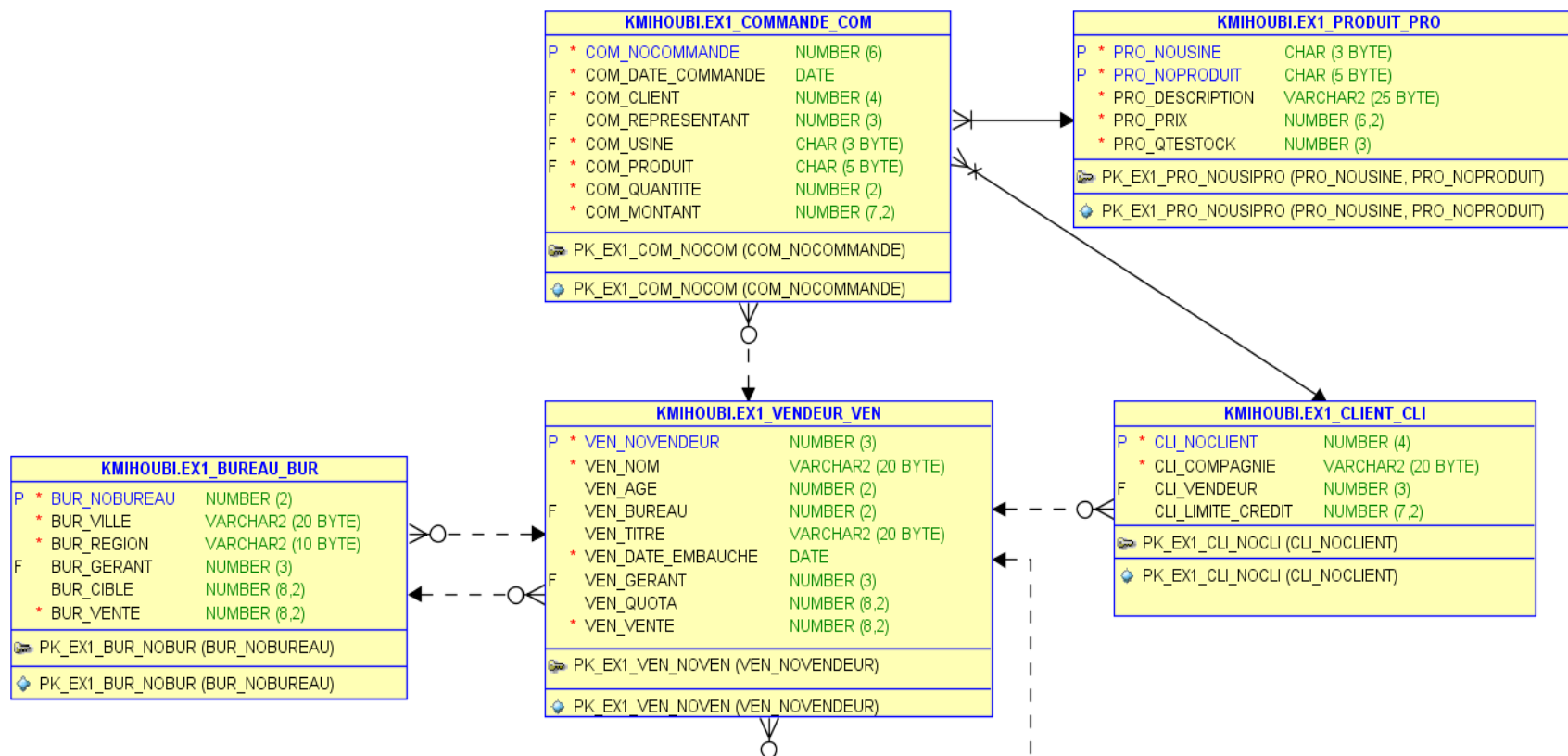
```
SELECT VEN_NOM, VEN_BUREAU  
FROM EX1_VENDEUR_VEN;
```

VEN_NOM	VEN_BUREAU
-----	-----
Samuel Cleroux	11
Robert Smith	12
Benoit Legault	13
Daniel Robert	12
Jacques Lacourse	
Paul Guidon	12
Lionel Marchand	21
Suzanne Labonte	21
Nancy Fafard	22
Marie Juillet	11



Jointure externe

ORACLE





Jointure externe

ORACLE

Si on construit la requête avec la table **EX1_BUREAU_BUR**, on pourra traduire les numéros de bureau en nom de ville, mais...

```
SELECT VEN_NOM, BUR_VILLE  
FROM EX1_VENDEUR_VEN INNER JOIN EX1_BUREAU_BUR  
ON VEN_BUREAU = BUR_NOBUREAU
```

**L'uplet de Jacques Lacourse
a disparu!**

**Normal, à cause de la
jointure interne...**

VEN_NOM	BUR_VILLE
-----	-----
Samuel Cleroux	Montreal
Robert Smith	Toronto
Benoit Legault	Ottawa
Daniel Robert	Toronto
Paul Guidon	Toronto
Lionel Marchand	Calgary
Suzanne Labonte	Calgary
Nancy Fafard	Vancouver
Marie Juillet	Montreal



Jointure externe

ORACLE

- On peut forcer l'**uplet de Jacques Lacourse** à réapparaître dans la sortie en utilisant une jointure externe!
- Ce que l'on veut, c'est conserver tous les uplets de la **table principale** ayant une correspondance **NULL** dans la **table secondaire**
- Dans ce cas-ci c'est la table **EX1_VENDEUR_VEN** qui devient la **table principale** et la table **EX1_BUREAU_BUR** devient la **table secondaire**



Jointure externe

ORACLE

On met à gauche(**LEFT**) ou à droite (**RIGHT**). On force l'**uplet de Jacques Lacourse** à réapparaître dans la sortie en utilisant une **jointure externe**!

```
SELECT VEN_NOM, COALESCE(BUR_VILLE, 'N/A') AS VILLE  
FROM EX1_VENDEUR_VEN LEFT OUTER JOIN EX1_BUREAU_BUR  
ON VEN_BUREAU = BUR_NOBUREAU
```

et revoilà Jacques!

VEN_NOM	VILLE
Nancy Fafard	Vancouver
Marie Juillet	Montreal
Samuel Cleroux	Montreal
Paul Guidon	Toronto
Daniel Robert	Toronto
Robert Smith	Toronto
Benoit Legault	Ottawa
Suzanne Labonte	Calgary
Lionel Marchand	Calgary
Jacques Lacourse	N/A



Jointure externe

ORACLE

- MÊME EXEMPLE AVEC L'ANCIENNE ÉCRITURE!

- En *SQL SERVER*, on peut écrire :

```
SELECT VEN_NOM, BUR_VILLE  
FROM EX1_VENDEUR_VEN, EX1_BUREAU_BUR  
WHERE VEN_BUREAU *= BUR_NOBUREAU
```

- En *SQL SERVER*, on place le (*) du côté de la **table principale**, soit celle qui contient tous les uplets

- En *Oracle*, on peut écrire :

```
SELECT VEN_NOM, BUR_VILLE  
FROM EX1_VENDEUR_VEN, EX1_BUREAU_BUR  
WHERE VEN_BUREAU = BUR_NOBUREAU (+)
```

- En *Oracle*, on place le (+) du côté de la **table dominée**, soit celle qui contient les uplets **NULLS**



Jointure externe

ORACLE

- Si, dans une deuxième situation, on cherche *les bureaux qui n'ont pas de vendeurs*, alors...
- Ce que l'on veut, c'est conserver tous les uplets de la *table principale* ayant une correspondance *NULL* dans la *table secondaire*
- Dans ce cas-ci c'est la table *EX1_BUREAU_BUR* qui devient la *table principale* et la table *EX1_VENDEUR_VEN* devient la *table secondaire*



Jointure externe

ORACLE

- Cherchons les *bureaux qui n'ont pas de vendeur!*

```
SELECT VEN_NOM, BUR_VILLE  
FROM EX1_VENDEUR_VEN RIGHT OUTER JOIN EX1_BUREAU_BUR  
ON VEN_BUREAU = BUR_NOBUREAU
```

VEN_NOM	BUR_VILLE
-----	-----
Samuel Cleroux	Montreal
Robert Smith	Toronto
Benoit Legault	Ottawa
Daniel Robert	Toronto
Paul Guidon	Toronto
Lionel Marchand	Calgary
Suzanne Labonte	Calgary
Nancy Fafard	Vancouver
Marie Juillet	Montreal

Note :

Tous les bureaux ont
au moins un vendeur...



Jointure externe

ORACLE

- MÊME EXEMPLE AVEC L'ANCIENNE ÉCRITURE!

- En **SQL SERVER**, on peut écrire :

```
SELECT VEN_NOM, BUR_VILLE  
FROM EX1_VENDEUR_VEN, EX1_BUREAU_BUR  
WHERE VEN_BUREAU =* BUR_NOBUREAU
```

- En **SQL SERVER**, on place le (*) du côté de la **table principale**, soit celle qui contient tous les uplets

- En **Oracle**, on peut écrire :

```
SELECT VEN_NOM, BUR_VILLE  
FROM EX1_VENDEUR_VEN, EX1_BUREAU_BUR  
WHERE VEN_BUREAU (+) = BUR_NOBUREAU
```

- En **Oracle**, on place le (+) du côté de la **table secondaire**, soit celle qui contient les uplets **NULLS**



Jointure externe

ORACLE

- *Notre employeur souhaite retrouver les noms des clients qui n'ont passé aucune commande!*
- Naturellement ces données n'existent pas dans la table **EX1_COMMANDE_COM** car cette dernière ne contient que les clients de la table **EX1_CLIENT_CLI** qui ont passé une commande
- Une **jointure externe** va nous permettre de trouver la solution à cette requête!!!



Jointure externe

ORACLE

- Si nous faisons une *jointure externe* entre tous les clients de la table **EX1_CLIENT_CLI** (*table principale*) et toutes les commandes de la table **EX1_COMMANDE_COM** (*table secondaire*), nous verrons apparaître des **NULLS** du côté de la *table secondaire*
- Ensuite, il ne restera qu'à extraire les uplets pour lesquels il y aura des **NULLS** dans la colonne « **COM_NOCOMMANDE** » de la table résultante



Jointure externe

ORACLE

```
SELECT CLI_COMPAGNIE, COALESCE(TO_CHAR(COM_NOCOMMANDE), 'Aucune commande') AS COMMANDE
FROM EX1_CLIENT_CLI LEFT OUTER JOIN EX1_COMMANDE_COM
ON CLI_NOCLIENT = COM_CLIENT
WHERE COM_NOCOMMANDE IS NULL
```

```
SELECT CLI_COMPAGNIE, COM_NOCOMMANDE
FROM EX1_CLIENT_CLI LEFT OUTER JOIN EX1_COMMANDE_COM
ON CLI_NOCLIENT = COM_CLIENT
```

CLI_COMPAGNIE	COM_NOCOMMANDE
J.P. Sinclair	112961
JCP Inc.	113012
Jones Mfg.	112989
Systemes du MidWest	113051
Cie. Premiere	112968
Acura International	113036
Zetacorp	113045
Systemes du MidWest	113045
Acme Mfg.	112987
JCP Inc.	113057
Ian et Schmidt	113042
QMA Assoc.	
Solomon Inc.	
Cartier et Fils	
AAA Investissements	
Smithson Corp.	
Lignes Triaxiales	

CLI_COMPAGNIE	COMMANDE
QMA Assoc.	Aucune commande
Solomon Inc.	Aucune commande
Cartier et Fils	Aucune commande
AAA Investissements	Aucune commande
Smithson Corp.	Aucune commande
Lignes Triaxiales	Aucune commande

**Maintenant, voici les clients
qui n'ont passé aucune
commande...**

Voici tous les clients...



Jointure avec conditions

Équijointure

ORACLE

- Résumé : quand la ou les conditions de jointure sont construites sur la base d'une égalité ou d'une inégalité entre colonnes(le plus souvent entre une clé primaire et une clé étrangère), il s'agit alors d'une équijointure. On retrouve dans ce cas :
 - Jointure interne (**JOIN ON**)
 - Jointure interne (**JOIN USING**)
 - Jointure interne (**NATURAL JOIN**)
 - Auto-jointure (**SELF JOIN**)
 - Jointure externe (**OUTER JOIN**)



Jointure interne (Inéquijointure)

ORACLE

- Les requêtes d'inéquijointures font intervenir tout type d'opérateur (<>, >, <=, BETWEEN, LIKE, IN, etc.). À l'inverse des équijointures, la clause d'égalité n'est pas basée sur l'égalité de clés primaires (ou candidates) et de clés étrangères.

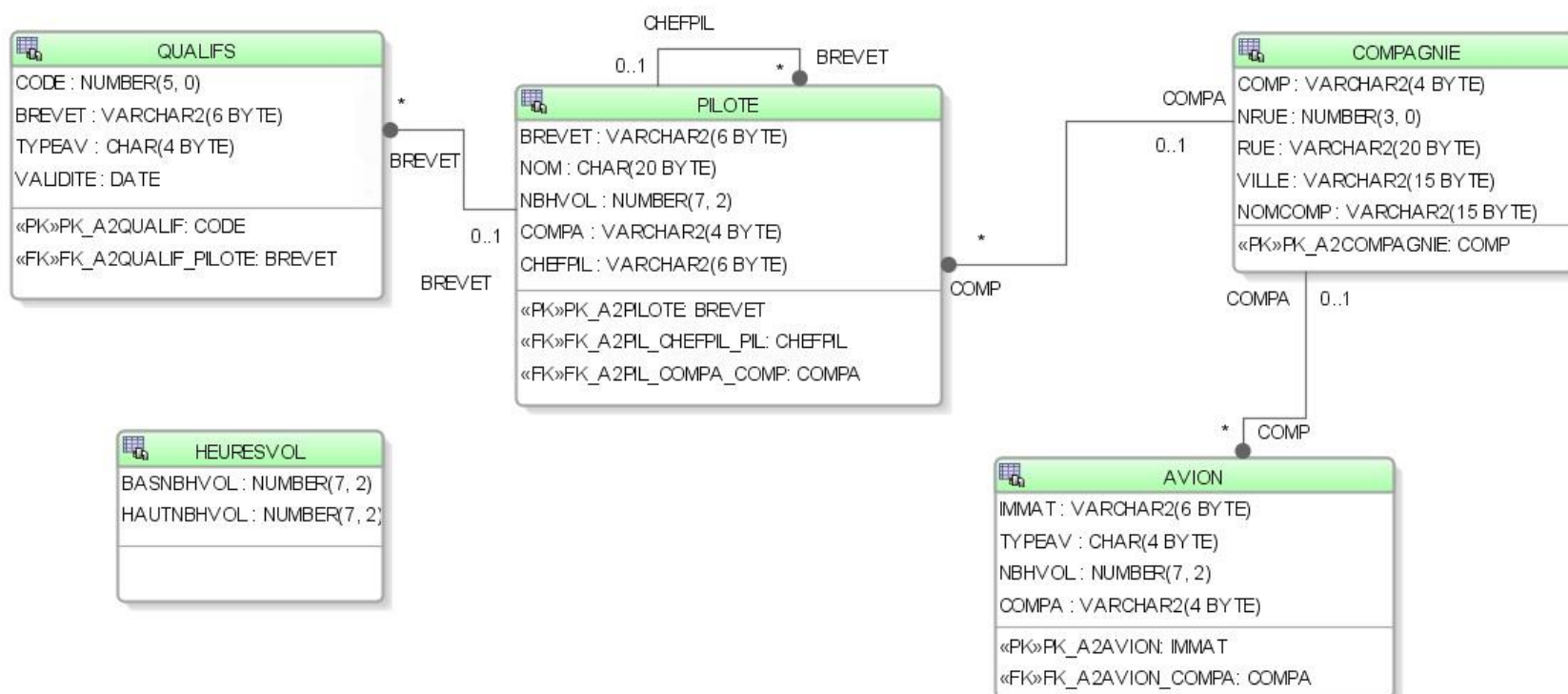
```
SELECT pil.brevet, pil.nom, pil.nbHVol, hv.titre
FROM Pilote pil, HeuresVol hv
WHERE pil.nbHVol BETWEEN
      hv.basnbHVol AND hv.hautnbHVol;
```

```
SELECT brevet, nom, nbHVol, titre
FROM Pilote INNER
      JOIN HeuresVol ON nbHVol
      BETWEEN basnbHVol AND hautnbHVol;
```



MLD Pilote

ORACLE





Jointure interne (Inéquijointure)

ORACLE

- **Deux styles d'écriture** : le titre de qualification des pilotes en raisonnant sur la comparaison des heures de vol avec un ensemble de référence. Il s'agit ici de retrouver le niveau d'expérience du pilote en fonction du nbHeures.

```
SELECT pil.brevet, pil.nom, pil.nbHVol, hv.titre
FROM Pilote pil, HeuresVol hv
WHERE pil.nbHVol BETWEEN
      hv.basnbHVol AND hv.hautnbHVol;
```

```
SELECT brevet, nom, nbHVol, titre
FROM Pilote INNER
      JOIN HeuresVol ON nbHVol
      BETWEEN basnbHVol AND hautnbHVol;
```



Jointure interne (Inéquijointure)

ORACLE

```
1  /* Toutes les commandes produits dont le prix unitaire produit commandé est supérieure
2  à celui du prix unitaire produit de code 2. */
3
4  SELECT DC.REF_PRODUIT PRODUIT, COUNT(DISTINCT NO_COMMANDE) CDE
5  FROM DETAILS_COMMANDES DC, PRODUITS PROD
6  WHERE DC.PRIX_UNITAIRE > PROD.PRIX_UNITAIRE AND PROD.REF_PRODUIT = 2
7  GROUP BY DC.REF_PRODUIT;
```

PR...	PRIX_UNITAIRE	CDE
1	120	95,4 3799
2	20	95,16 4133
3	96	95,52 4271

Trois produits ayant été commandé ont un prix unitaire supérieur à celui du produit de code 2 qui est de 95\$.



Jointures

ORACLE

EN RÉSUMÉ...

- La **nouvelle nomenclature** des *jointures* utilise
 - *Jointure interne*
SELECT ...
FROM Table-gauche INNER JOIN **Table-droite**
ON ...
 - *Jointure externe*
SELECT ...
FROM Table-gauche { LEFT | RIGHT | FULL } OUTER JOIN
Table-droite
ON ...



Jointures

ORACLE

EN RÉSUMÉ...

- Vous êtes quand même susceptible de rencontrer encore *l'ancienne écriture*, alors attention à la syntaxe!!!
 - **Jointure interne**

```
SELECT ...  
FROM Table-gauche, Table-droite  
WHERE ...
```
 - **Jointure externe**

<pre>(SQL SERVER) SELECT ... FROM Table-gauche, Table-droite WHERE A *= B</pre>	<pre>(ORACLE) SELECT ... FROM Table-gauche, Table-droite WHERE A = B(+)</pre>
---	---
- En **SQL**, on place le ***** du côté de la table où on veut conserver tous les uplets, soit la table « principale »
- En **Oracle**, on place le **(+)** du côté de la table qui contient les NULLS, soit la table « secondaire »



EXERCICES

ORACLE

